



本小节内容

二叉树的层序遍历实战

二叉树的层序遍历实战

在 14.4 我们实战了二叉树层次建树，层次遍历与层次建树的原理非常类似。

层次遍历我们必须使用辅助队列，为了提高代码的编写效率，我们可以将 13.7 小节的队列实战代码直接拿过来，变为 queue.cpp 来使用，由于 main.cpp 和 queue.cpp 都需要使用到对应的结构体类型，所以我们把结构体类型的声明放入 function.h 头文件，function.h 头文件在 main.cpp 和 queue.cpp 中都进行了 include，下面直接上代码：

function.h 代码如下

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
typedef char BiElemType;
```

```
typedef struct BiTNode{
```

```
    BiElemType c;//c 就是书籍上的 data
```

```
    struct BiTNode *lchild;
```

```
    struct BiTNode *rchild;
```

```
}BiTNode,*BiTree;
```

```
typedef struct tag{
```

```
    BiTree p;//树的某一个结点的地址值
```

```
    struct tag *pnext;
```

```
}tag_t,*ptag_t;
```

```
//队列的相关数据结构
```

```
typedef BiTree ElemType;
```

```
typedef struct LinkNode{
```

```
    ElemType data;
```

```
    struct LinkNode *next;
```

```
}LinkNode;
```

```
typedef struct{
```

```
    LinkNode *front,*rear;
```

```
}LinkQueue;
```

```
void InitQueue(LinkQueue &Q);
```

```
bool IsEmpty(LinkQueue Q);
```



```
void EnQueue(LinkQueue &Q, ElemType x);  
bool DeQueue(LinkQueue &Q, ElemType &x);
```

queue.cpp 代码如下:

```
#include "function.h"  
void InitQueue(LinkQueue &Q)  
{  
    Q.front=Q.rear=(LinkNode*)malloc(sizeof(LinkNode)); //头和尾指向同一个结点  
    Q.front->next=NULL; //头结点的 next 指针为 NULL  
}  
  
bool IsEmpty(LinkQueue Q)  
{  
    if(Q.front==Q.rear)  
        return true;  
    else  
        return false;  
}  
//入队，尾部插入法  
void EnQueue(LinkQueue &Q, ElemType x)  
{  
    LinkNode *s=(LinkNode *)malloc(sizeof(LinkNode));  
    s->data=x; s->next=NULL;  
    Q.rear->next=s; //rear 始终指向尾部  
    Q.rear=s;  
}  
//出队 头部删除法  
bool DeQueue(LinkQueue &Q, ElemType &x)  
{  
    if(Q.front==Q.rear) return false; //队列为空  
    LinkNode *p=Q.front->next; //头结点什么都没有存，所以头结点的下一个节点才有数据  
    x=p->data;  
    Q.front->next=p->next; //断链  
    if(Q.rear==p) //删除的是最后一个元素  
        Q.rear=Q.front; //队列置为空  
    free(p);  
    return true;  
}
```

下面是 main.cpp 的代码，我们在上一小节的基础上增加了 LevelOrder 函数，重

点看 LevelOrder 即可

```
#include "function.h"
```



//递归实现

//abdhiejc fg 前序遍历，前序遍历就是深度优先遍历

void PreOrder(BiTree p)

```
{
    if(p!=NULL)
    {
        putchar(p->c); //等价于 visit 函数
        PreOrder(p->lchild);
        PreOrder(p->rchild);
    }
}
```

//中序遍历 h d i b j e a f c g

void InOrder(BiTree p)

```
{
    if(p!=NULL)
    {
        InOrder(p->lchild);
        putchar(p->c);
        InOrder(p->rchild);
    }
}
```

//h i d j e b f g c a 后序遍历

void PostOrder(BiTree p)

```
{
    if(p!=NULL)
    {
        PostOrder(p->lchild);
        PostOrder(p->rchild);
        putchar(p->c);
    }
}
```

//层次遍历,层序遍历，广度优先遍历

void LevelOrder(BiTree T)

```
{
    LinkQueue Q; //辅助队列
    InitQueue(Q); //初始化队列
    BiTree p;
    EnQueue(Q,T); //树根入队
    while(!IsEmpty(Q))
    {
        DeQueue(Q,p); //出队当前结点并打印
        putchar(p->c);
        if(p->lchild!=NULL) //入队左孩子
            EnQueue(Q,p->lchild);
    }
}
```



```
        if(p->rchild!=NULL) //入队右孩子
            EnQueue(Q,p->rchild);
    }
}
//《王道 C 督学营》课程
//二叉树的建树（层次建树）
int main()
{
    BiTree pnew;//用来指向新申请的树结点
    char c;
    BiTree tree=NULL;//树根
    //phead 就是队列头，ptail 就是队列尾
    ptag_t phead=NULL,ptail=NULL,listpnew=NULL,pcur=NULL;
    //输入内容为 abcdefghij
    while(scanf("%c",&c))
    {
        if(c=='\n')
        {
            break;
        }
        pnew=(BiTree)calloc(1,sizeof(BiTNode));//calloc 申请空间并对空间进行初始化，赋值为 0
        pnew->c=c;//数据放进去
        listpnew=(ptag_t)calloc(1,sizeof(tag_t));//给队列结点申请空间
        listpnew->p=pnew;
        if(NULL==tree)
        {
            tree=pnew;//树的根
            phead=listpnew;//队列头
            ptail=listpnew;//队列尾
            pcur=listpnew;
            continue;
        }else{
            ptail->pnext=listpnew;//新结点放入链表，通过尾插法
            ptail=listpnew;//ptail 指向队列尾部
        }//pcur 始终指向要插入的结点的位置
        if(NULL==pcur->p->lchild)//如何把新结点放入树
        {
            pcur->p->lchild=pnew;//把新结点放到要插入结点的左边
        }else if(NULL==pcur->p->rchild)
        {
            pcur->p->rchild=pnew;//把新结点放到要插入结点的右边
            pcur=pcur->pnext;//左右都放了结点后，pcur 指向队列的下一个
        }
    }
}
```



```
printf("-----前序遍历-----\n");//也叫先序遍历，先打印当前结点，打印左孩子，打  
印右孩子  
PreOrder(tree);  
printf("\n-----中序遍历-----\n");//先打印左孩子，打印父亲，打印右孩子  
InOrder(tree);  
printf("\n-----后序遍历-----\n");//先打印左孩子，打印右孩子，最后打印父亲  
PostOrder(tree);  
printf("\n-----层次遍历-----\n");  
LevelOrder(tree);  
printf("\n");  
return 0;  
}
```